

ARCHITECTURE DES SYSTÈMES D'INFORMATION

École d'ingénieurs — Année 2025-2026

Projet Microservices

Application de Location de Logement

Groupe :

Prénom Nom — Prénom Nom

Prénom Nom — Prénom Nom

Avril 2026

Table des matières

1	Introduction	2
2	Conception de l'application	2
2.1	Réflexion sur le découpage	2
2.2	Vue d'ensemble de l'architecture	2
2.3	Description des services	3
2.3.1	API Gateway	3
2.3.2	Auth-service	3
2.3.3	User-service	3
2.3.4	Property-service	4
2.3.5	Messaging-service	4
2.3.6	Interfaces frontend	4
2.4	Interactions entre les services	4
2.5	Gestion de la résilience	6
3	Technologies adoptées	7
3.1	Stack technique	7
3.2	Gestion des interactions entre les services	8
3.3	Gestion de la configuration	8
3.4	Gestion de l'authentification et des habilitations	9
3.5	Gestion du déploiement	9
3.6	Gestion des erreurs et logging	10
4	Fonctionnalités réalisées	10
4.1	Côté locataire	10
4.2	Côté propriétaire	12
4.3	Résilience	13
5	Conclusion	14

1 Introduction

Ce rapport présente la conception et la réalisation d'une application de location de logement basée sur une architecture microservices. Le projet a été réalisé dans le cadre du cours d'Architecture des Systèmes d'Information.

L'objectif principal est de mettre en pratique les concepts vus en cours en construisant une application fonctionnelle où chaque service a une responsabilité claire et peut fonctionner de manière indépendante. Nous avons choisi le cas d'usage d'une plateforme de location (comparable à Airbnb ou Booking) car il se prête bien au découpage en microservices : la gestion des utilisateurs, des logements, des réservations et de la messagerie sont des domaines métier distincts.

Ce document détaille notre réflexion sur le découpage des services, les interactions entre eux, les choix technologiques et la stratégie de déploiement.

2 Conception de l'application

2.1 Réflexion sur le découpage

Avant de commencer le développement, nous avons identifié les différents domaines métier de l'application. L'idée était de séparer les responsabilités pour que chaque service gère un périmètre fonctionnel précis. Nous avons retenu le principe suivant : si un domaine peut fonctionner (ou échouer) indépendamment des autres, alors il mérite son propre service.

Par exemple, si le service de messagerie tombe en panne, les utilisateurs doivent pouvoir continuer à naviguer sur les logements et à effectuer des réservations. De même, si le service d'authentification est temporairement indisponible, les logements restent consultables sans connexion. Ce raisonnement nous a conduits à identifier six services backend et deux interfaces frontend.

Nous avons aussi fait le choix de séparer les données : chaque service possède sa propre base de données. Le user-service a sa base PostgreSQL pour les utilisateurs, le property-service a la sienne pour les logements et réservations, et le messaging-service a la sienne pour les messages. Cette séparation empêche un couplage fort entre les services au niveau des données.

2.2 Vue d'ensemble de l'architecture

Notre application se compose de six services backend, deux interfaces frontend, trois bases de données PostgreSQL et un service de stockage d'objets (MinIO). Un point d'entrée unique, l'API Gateway, expose l'ensemble de l'application au monde extérieur.

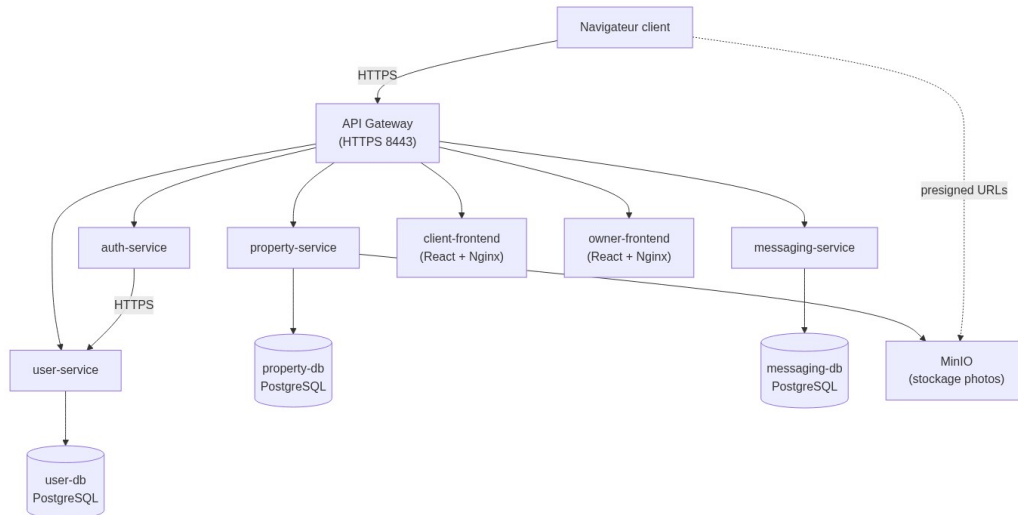


FIGURE 1 – Architecture globale de l'application

2.3 Description des services

2.3.1 API Gateway

L'API Gateway est le seul point d'entrée exposé à l'extérieur. Tous les appels passent par elle, qu'il s'agisse de requêtes API ou de pages frontend. Son rôle est triple : router les requêtes vers le bon service, valider les tokens JWT pour les routes protégées, et servir les deux interfaces frontend.

Nous avons fait le choix d'un point d'entrée unique car cela simplifie la configuration réseau et la sécurité. Les services internes n'ont pas besoin d'être exposés directement, ce qui réduit la surface d'attaque. Si un service tombe, la gateway retourne une erreur 503 avec un message clair au lieu de laisser le client face à une erreur réseau incompréhensible.

La gateway route les requêtes selon le préfixe de l'URL. Les appels commençant par `/api/auth/` sont dirigés vers le auth-service, ceux par `/api/properties/` vers le property-service, etc. Les requêtes vers `/owner/` servent l'interface propriétaire et toutes les autres servent l'interface locataire.

2.3.2 Auth-service

Le auth-service gère l'authentification des utilisateurs. Il expose deux endpoints : l'inscription et la connexion. Lors de la connexion, il vérifie les identifiants en appelant le user-service, puis génère un token JWT si les identifiants sont corrects.

Nous avons séparé l'authentification dans un service dédié car c'est une responsabilité transverse qui ne concerne ni la gestion des logements ni celle des utilisateurs à proprement parler. Le auth-service ne possède pas de base de données propre : il délègue le stockage des données utilisateur au user-service. Ce choix évite la duplication de données.

2.3.3 User-service

Le user-service est responsable du stockage et de la gestion des données utilisateur (email, mot de passe hashé, rôle, nom, téléphone). Il expose des endpoints CRUD classiques et sert de source de vérité pour les informations des comptes.

La séparation entre user-service et auth-service permet une chose importante : si le service d'authentification est en panne, les informations utilisateur restent accessibles aux autres services qui en ont besoin. Par exemple, le property-service peut afficher le nom d'un propriétaire sans dépendre du auth-service.

2.3.4 Property-service

Le property-service est le service métier principal. Il gère les logements (création, modification, recherche avec filtres) et les réservations (création avec vérification de disponibilité, confirmation, annulation). Il gère aussi les photos des logements via MinIO.

Nous avons regroupé les logements et les réservations dans le même service car ces deux domaines sont fortement couplés : une réservation est toujours liée à un logement, et la recherche de logements prend en compte les réservations existantes pour filtrer par disponibilité. Séparer ces deux aspects aurait engendré beaucoup de communications inter-services pour des opérations simples.

Pour les photos, le property-service ne stocke pas les fichiers. Il génère des URLs pré-signées via MinIO : le navigateur du client envoie les photos directement vers MinIO et les récupère directement depuis MinIO. Le service ne fait que stocker les clés d'objets en base de données. Ce choix allège le service et évite de faire transiter des fichiers volumineux par le backend.

2.3.5 Messaging-service

Le messaging-service gère les conversations entre locataires et propriétaires. Une conversation est liée à une réservation confirmée. Chaque message contient l'identifiant de l'expéditeur, son rôle (locataire ou propriétaire), le contenu et un indicateur de lecture.

Nous avons fait le choix d'un service séparé pour la messagerie car c'est une fonctionnalité indépendante du reste. Si ce service tombe en panne, les utilisateurs peuvent toujours chercher des logements, réserver et gérer leurs propriétés. C'est un bon exemple de la résilience que permet l'architecture microservices. La messagerie n'est accessible que pour les réservations confirmées, ce qui lie fonctionnellement les deux services sans les coupler techniquement.

2.3.6 Interfaces frontend

Nous avons deux interfaces distinctes : une pour les locataires et une pour les propriétaires. Les locataires accèdent à la recherche de logements, la réservation et la messagerie. Les propriétaires accèdent à la gestion de leurs logements, au suivi des demandes de réservation et à la messagerie.

Les deux frontends sont des applications React servies par Nginx. Elles sont servies par la gateway : l'interface locataire est accessible sur / et l'interface propriétaire sur /owner/. Ce sont deux applications séparées car les besoins sont différents, mais elles partagent le même design system pour la cohérence visuelle.

2.4 Interactions entre les services

Les services communiquent entre eux uniquement via des appels HTTP REST en HTTPS. Nous n'avons pas utilisé de message broker (comme RabbitMQ) car les interactions sont synchrones et simples pour notre cas d'usage.

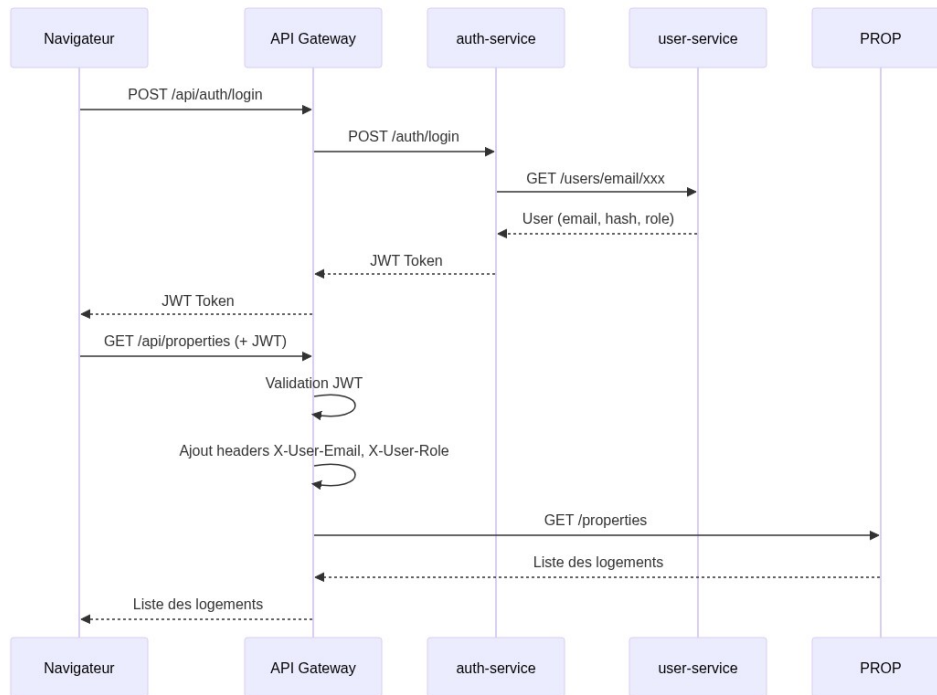


FIGURE 2 – Flux d’authentification et de routage via la gateway

Appelant	Appelé	Raison
Navigateur	API Gateway	Toutes les requêtes passent par la gateway
API Gateway	Tous les services	Routage des requêtes selon le préfixe URL
auth-service	user-service	Vérification des identifiants et création de comptes
Navigateur	MinIO	Upload et téléchargement des photos (via URLs pré-signées)

TABLE 1 – Interactions entre les services

Un point important : les services backend ne s’appellent pas directement entre eux (sauf auth-service qui appelle user-service). Le property-service et le messaging-service sont indépendants. La coordination est faite côté client : par exemple, le frontend appelle le property-service pour les détails d’un logement et le messaging-service pour les messages, puis assemble les données.

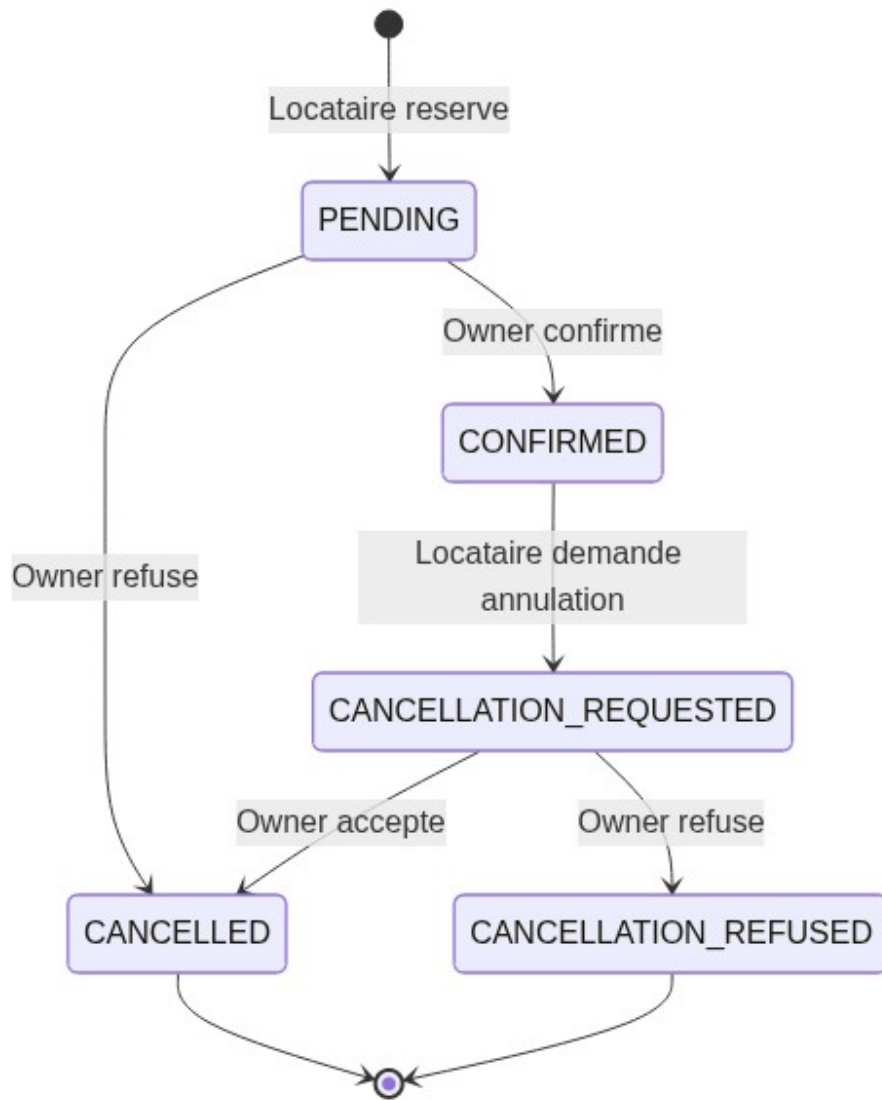


FIGURE 3 – Cycle de vie d’une réservation

2.5 Gestion de la résilience

Un des objectifs du projet était de montrer que l’application continue de fonctionner même quand certains services sont en panne. Nous avons mis en place plusieurs mécanismes pour cela.

Au niveau de la gateway, un gestionnaire d’erreurs intercepte les erreurs de connexion vers les services internes et retourne une réponse JSON avec un code 503 et le nom du service concerné. Le frontend peut alors afficher un message clair à l’utilisateur.

Au niveau des frontends, chaque page qui appelle un service backend gère le cas où ce service est indisponible. Un composant `ServiceUnavailable` s’affiche avec un bouton "Réessayer" au lieu de laisser l’utilisateur face à une page blanche.

Nous avons aussi créé une page de statut accessible sur `/status` qui affiche en temps réel l’état de chaque service avec des pastilles vertes ou rouges. Cette page est utile pour la démonstration.

Capture : page de statut des services (tous les services actifs)

FIGURE 4 – Page de statut des services

Capture : page d'accueil quand le property-service est arrêté

FIGURE 5 – Mode dégradé — property-service arrêté

3 Technologies adoptées

3.1 Stack technique

Composant	Technologie
Services backend	Java 17, Spring Boot 3.2.5
Gateway	Spring Cloud Gateway 2023.0.1
Authentification	JWT (bibliothèque jjwt 0.12.5)
Bases de données	PostgreSQL 15
Stockage photos	MinIO (compatible S3)
Frontends	React 18, Vite, React Router
Conteneurisation	Docker, Docker Compose
Communication	HTTPS avec certificats auto-signés

TABLE 2 – Stack technique du projet

Nous avons choisi Java et Spring Boot car c'est la stack vue en cours et la plus utilisée pour les microservices dans l'industrie. Spring Cloud Gateway fournit un mécanisme de

routing déclaratif qui nous a permis de configurer toutes les routes dans un fichier YAML sans écrire de logique complexe.

Pour les frontends, nous avons choisi React avec Vite pour sa simplicité de mise en place et sa popularité. Les applications sont construites en images Docker multi-stage (build avec Node.js, puis servies par Nginx).

3.2 Gestion des interactions entre les services

Toutes les communications entre services se font en HTTPS avec des certificats auto-signés. Un script `generate-certs.sh` génère une autorité de certification locale, un certificat serveur, et un keystore PKCS12 partagé entre tous les services Spring Boot. Les services se font confiance via le flag `useInsecureTrustManager` de la gateway.

Les appels inter-services utilisent RestTemplate (dans le cas du auth-service vers le user-service) avec une configuration SSL qui accepte les certificats auto-signés. Ce n'est pas une pratique de production, mais c'est suffisant pour démontrer le principe du chiffrement des communications.

Les frontends communiquent uniquement avec la gateway via des appels REST en JSON. L'API est préfixée par `/api/` pour distinguer les routes API des routes frontend.

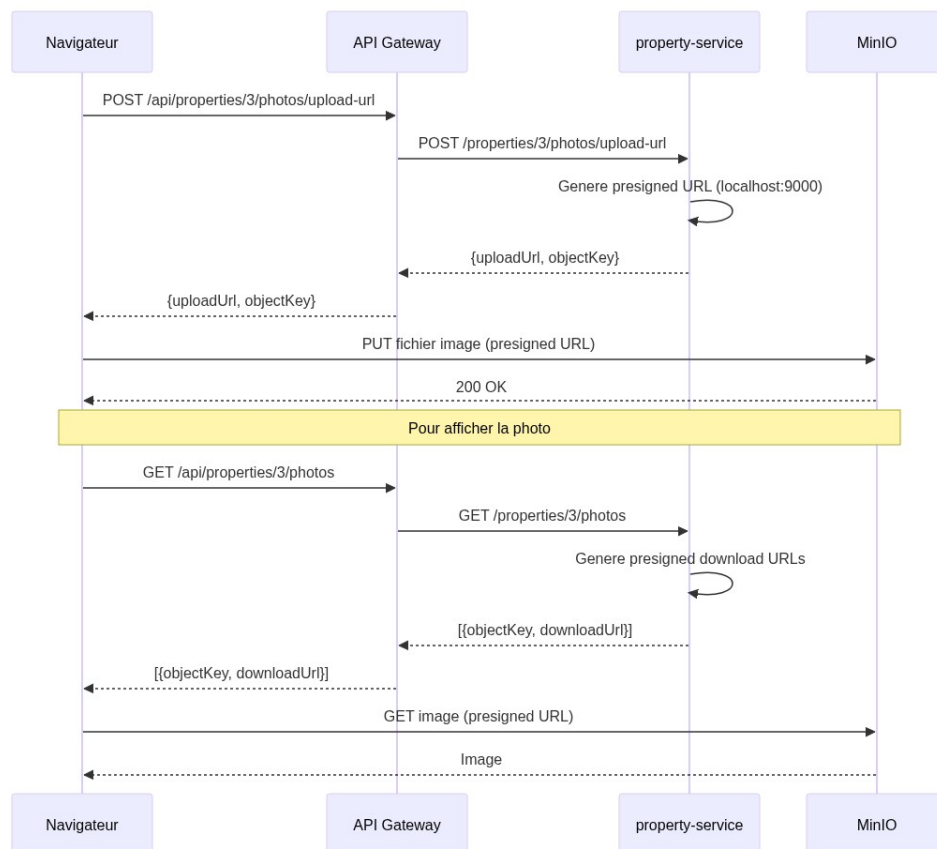


FIGURE 6 – Flux d'upload et de téléchargement des photos via MinIO

3.3 Gestion de la configuration

La configuration de chaque service est centralisée dans un fichier `application.yml`. Les valeurs sensibles ou spécifiques à l'environnement (URL des bases de données, clé

secrète JWT, credentials MinIO) sont externalisées via des variables d'environnement, définies dans le `docker-compose.yml`.

Par exemple, la clé secrète JWT est définie comme `${JWT_SECRET:-valeur_defaut}` dans la configuration : elle utilise une valeur par défaut pour le développement local mais peut être remplacée en production sans modifier le code.

Nous n'avons pas mis en place de serveur de configuration centralisé (comme Spring Cloud Config) car pour notre nombre de services, la gestion par variables d'environnement est suffisante.

3.4 Gestion de l'authentification et des habilitations

L'authentification repose sur des tokens JWT. Lorsqu'un utilisateur se connecte, le auth-service génère un token contenant l'email et le rôle de l'utilisateur (locataire ou propriétaire). Ce token a une durée de validité de 24 heures.

La gateway vérifie le token sur toutes les routes protégées. Les routes publiques (consultation des logements, inscription, connexion) ne nécessitent pas de token. Quand le token est valide, la gateway ajoute les en-têtes `X-User-Email` et `X-User-Role` à la requête transmise au service backend. Les services backend peuvent ainsi connaître l'identité de l'utilisateur sans valider le token eux-mêmes.

La clé secrète JWT est partagée entre le auth-service (qui signe les tokens) et la gateway (qui les vérifie). Elle est injectée via une variable d'environnement commune dans le Docker Compose.

3.5 Gestion du déploiement

Chaque service est conteneurisé avec Docker. Les Dockerfile suivent le même pattern multi-stage : compilation avec Maven dans une image de build, puis copie du JAR dans une image d'exécution légère (Alpine).

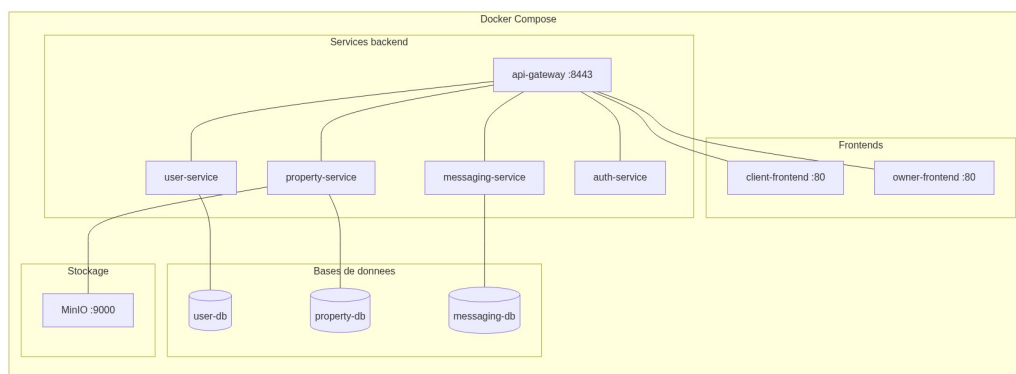


FIGURE 7 – Vue d'ensemble des conteneurs Docker Compose

L'ensemble de l'application est orchestré par Docker Compose. Un `Makefile` simplifie les commandes courantes :

<code>make up</code>	Génère les certificats, build et lance tous les services
<code>make down</code>	Arrête tous les services
<code>make reset-db</code>	Vide les bases de données et relance (les fixtures se rechargent)
<code>make logs</code>	Affiche les logs de tous les services
<code>make clean</code>	Supprime tout (conteneurs, volumes, certificats)

TABLE 3 – Commandes du Makefile

Le seul prérequis pour lancer l'application est Docker. Les certificats SSL sont générés dans un conteneur Alpine (pas besoin d'OpenSSL sur la machine hôte), les services Java sont compilés dans des conteneurs Maven, et les frontends sont construits dans des conteneurs Node.js.

Des données de démonstration (comptes utilisateurs, logements, réservations, messages) sont automatiquement insérées au démarrage de chaque service via des `CommandLineRunner` Spring Boot.

3.6 Gestion des erreurs et logging

Chaque service backend utilise un `@RestControllerAdvice` qui intercepte toutes les exceptions et retourne un format d'erreur JSON uniforme :

```
{
  "status": 404,
  "error": "Not Found",
  "message": "Logement non trouve : 42",
  "service": "property-service",
  "timestamp": "2026-04-07T12:00:00"
}
```

Ce format est le même pour tous les services, ce qui permet aux frontends de traiter les erreurs de manière homogène. Les exceptions custom (`EntityNotFoundException`, `ConflictException`) sont mappées vers les codes HTTP appropriés (404, 409).

Le logging utilise SLF4J (inclus dans Spring Boot). Chaque requête entrante et chaque erreur sont loggées avec le nom du service pour faciliter le diagnostic dans les logs Docker Compose.

4 Fonctionnalités réalisées

4.1 Côté locataire

L'interface locataire permet de rechercher des logements par ville, dates de disponibilité, type et prix maximum. Les résultats s'affichent sous forme de cartes avec la photo du logement. La page de détail permet de consulter les informations, les photos et de réserver en sélectionnant des dates. Les dates déjà réservées sont grisées dans le calendrier.

Le locataire peut suivre ses réservations (en attente, confirmée, annulée), demander une annulation, et communiquer avec le propriétaire via la messagerie une fois la réservation confirmée.



Capture : page d'accueil avec recherche de logements

FIGURE 8 – Page d'accueil — recherche de logements



Capture : page de détail d'un logement avec calendrier

FIGURE 9 – Page de détail d'un logement



Capture : page "Mes réservations" côté locataire

FIGURE 10 – Mes réservations — côté locataire



FIGURE 11 – Messagerie — côté locataire

4.2 Côté propriétaire

L'interface propriétaire donne accès à un tableau de bord avec la liste des logements. Le propriétaire peut ajouter, modifier et supprimer des logements, gérer les photos, et consulter les demandes de réservation. Il peut confirmer ou refuser une réservation, accepter ou refuser une demande d'annulation, et communiquer avec les locataires.

Un système de notifications indique le nombre de réservations en attente et de messages non lus directement dans la barre de navigation.

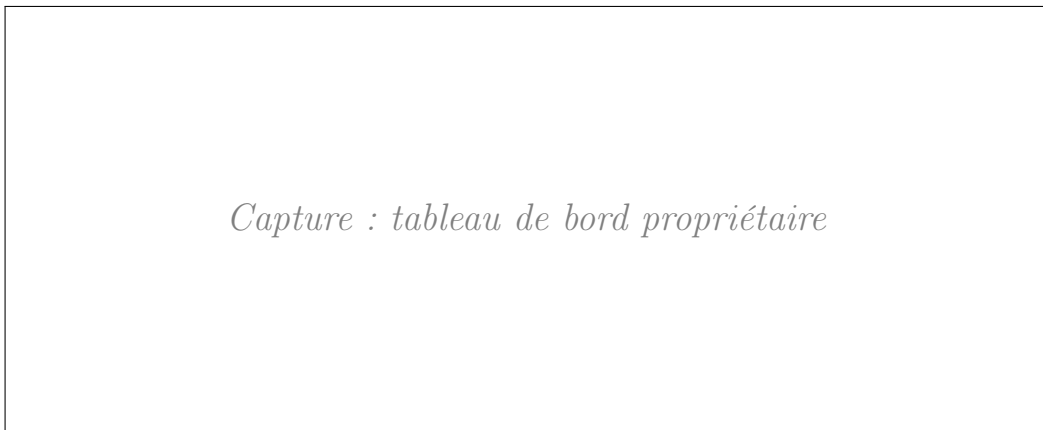


FIGURE 12 – Tableau de bord — côté propriétaire



FIGURE 13 – Réservations avec notifications — côté propriétaire



FIGURE 14 – Messagerie — côté propriétaire

4.3 Résilience

Quand un service backend est arrêté, l'application continue de fonctionner en mode dégradé. Les pages affichent un composant "Service temporairement indisponible" avec un bouton pour réessayer. Une page de statut montre l'état de chaque service en temps réel.

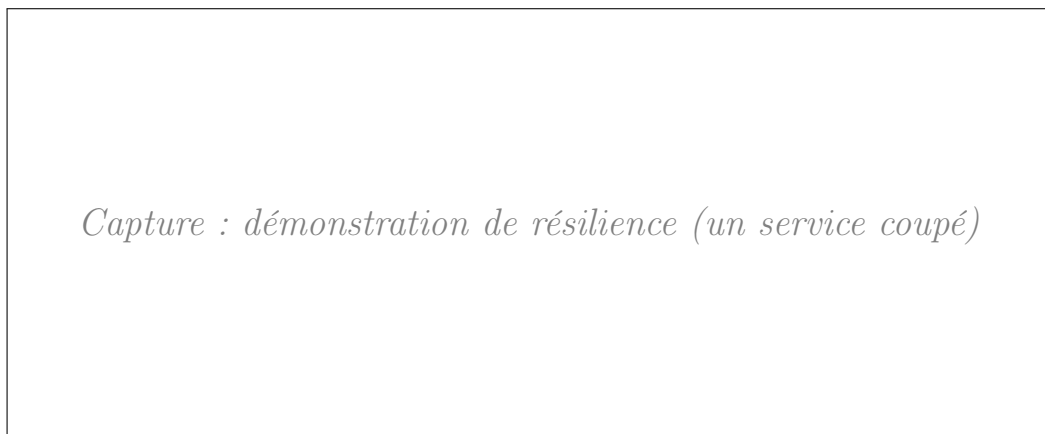


FIGURE 15 – Mode dégradé — démonstration de résilience

5 Conclusion

Ce projet nous a permis de mettre en pratique les concepts d'architecture microservices dans un cas concret. Le découpage en services indépendants, chacun avec sa propre base de données et sa propre responsabilité, nous a montré les avantages de cette approche : déploiement indépendant, résilience aux pannes, et séparation claire des responsabilités.

Nous avons aussi constaté les défis : la gestion de la communication HTTPS entre services, la propagation de l'authentification via JWT, et la complexité de l'infrastructure Docker Compose avec de nombreux conteneurs. Le projet final comporte six services backend, deux frontends, trois bases de données, un service de stockage et une gateway, soit plus de douze conteneurs Docker.

Parmi les améliorations possibles, nous pourrions ajouter un service de configuration centralisé (Spring Cloud Config), un service discovery (Eureka), du load balancing avec plusieurs instances de chaque service, ou un message broker pour les communications asynchrones. Ces ajouts seraient pertinents dans un contexte de production mais dépassent le cadre de ce projet éducatif.